REQUIREMENT SPECIFICATION

Dixen: Event Management, Discovery & Ticketing System



DIKSHYA SINGH SHAH

CLASS: 1205hf26016per

28. JAN 2026

TECHNICAL EDUCATION COPENHAGEN

Guldblommevej 5, Ballerup

Project Title: Requirement Specification

(Event Management, Discovery & Ticketing System)

Project Name: Dixen

Name: Dikshya Singh Shah

Date: 28-01-2026

School: TEC Ballerup

Supervisor: Flemming Sørensen

Signature fields

Signatures:

Student: _____ Date: _____

DIKSHYA SINGH SHAH

INDHOLDSFORTEGNELSE

1. Introduction 5
1.1 Purpose 5
1.2 scope 5
1.3 Stakeholders 5
1.4 Reading Guide 6
2. System overview 7
2.1 Product Perspective 7
2.2 Key Features 7
2.3 Program function 8
2.4 Operating Environment 8
2.5. development Methodology 8
3. User Requirements 9
3.1 User Profile 9
3.2 Users Expectations 9
4. Security 10
4.1 Authentication 10
4.2 Two-Factor Authentication (2FA) 10
4.3 Role-Based Access Control (RBAC) 10
4.4 Password & Session Management 12
4.5 Data Protection 12
5. Functional Requirements 13
5.1 Priority Defination 13
List of Requirements specification 13
User Registration and Login 13
Two-factor Authentication (2FA) 13
Email Communication 13
API Data Exchange 13
Event Management 13
Role-Based Access Control 14
Performer and Venue Management 14
Social Sharing 14
Analytics Dashboard 14

6 Use Case Descriptions 15
6.1 User Registration with Email confirmatiosn 15
6.2 Confirm Email Address 16
6.3 Login with 2FA (QR Code and Google Authentication) 17
6.4 Event Categorization (Events Can Belong to Multiple Categories) 18
6.5 Event Booking 19
6.6 Submit Event Proposal (EventSubmission) 20
6.7 Social Sharing (Share Event on Platforms Like Twitter, Facebook) 21
6.8 Event Management (Create, Edit, and Delete Events) 22
7. Non-Functional Requirements: 23
7.1 PERFORMANCE REQUIREMENTS 23
7.2 SECURITY REQUIREMENTS 23
7.3 USABILITY REQUIREMENTS 23
7.4 RELIABILITY AND AVAILABILITY 23
7.5 MAINTAINABILITY AND SCALABILITY 23
8. System Design 24
8.1 Frontend Structure (Angular) 24
8.2 Backend Structure (ASP.NET Web application) 24
9. Technical Product Documentation 24
9.1 Database Design 24
9.2 External Interfaces 24
9.3 Technologies Used 25
10. Authentication & Authorization Flow 25
10.1 User Registration (via /api/authentication/register): 25
10.2 Email Confirmation (via /api/authentication/confirm-email): 25
10.3 User Login (via /api/authentication/login): 25
10.4 Two-factor authentication (2FA) 26
11. Future Development & Integrations 26
12. Visual Disgrams 26
12.1. ER-Diagram 26
12.2 Use Case Diagram 28
12.3 Flow chart 29
12.3.1. User Registration 29
12.3.2. Email Verification 31

Login with 2FA 31
12.3.3. Login Process 32
12.3.4. Code Generation 33
12. References 34

1. INTRODUCTION

1.1 PURPOSE

This document defines the functional and non-functional requirements for Dixen, a modern, web-based event management platform. The system enables users to discover, propose, share events and manage events securely and efficiently. It supports multiple user roles, multi-slanguage functionality, and provides tools for event lifecycle management and analytics.

1.2 SCOPE

Dixen is a full-featured event management platform accessible via a modern web browser. The platform provides the following capabilities:

- Secure user registration and authentication mechanisms.
- Event proposal submission, approvals workflows.
- Role-based access for different user types.
- Event categorization and venue management.
- Admin Dashboard with analytic insights.
- Social media sharing integration promotes events.
- Multi-language support, allowing users to switch languages dynamically.

Out of scope:

- payment processing and push notifications.

1.3 STAKEHOLDERS

Guest Users: Visitors who can browse, search and view event details without registration.

End Users (Registered Users): Individuals who register, attend events, and propose events.

Hosts: Businesses or individuals responsible for creating and managing events.

Administrators: Platform administrators responsible for system configuration, approvals, and maintenance.

Support Team: Staff handling user support and technical issues.

1.4 READING GUIDE

This report provides a comprehensive overview of the Dixen Event Management, discovery and ticketing System, including its design, functionalities, and technical specifications. The sections are structured as follows:

Introduction: Explains the project's purpose, scope, and the stakeholders involved.

System Overview: Describes the overall architecture, program functions, and how the system operates.

User Requirements: Details user profiles, expectations, and how different user roles interact with the system.

Security: Details authentication methods, including two-factor authentication (2FA) and role-based access control (RBAC).

Functional Requirements: Covers key features such as user registration, event management, performer and venue management, and data exchange protocols.

Use Case Descriptions: Provides detailed use case scenarios describing user interactions with the system, including registration, login, event booking, proposal submission, and administrative actions.

Non-Functional Requirements: Specifies quality attributes including performance, security, usability, reliability, and maintainability.

System Design: Explains frontend (Angular) and backend (ASP.NET Core) structures, ER diagrams, and relevant use case scenarios.

Technical Product Documentation: Provides detailed system architecture, database design, and visual diagrams.

Authentication & Authorization Flow: Details user registration, email confirmation, login, and 2FA processes.

Future Development & Integrations: Outlines planned features like payment integration, push notifications, Google Calendar, and analytics dashboards.

Appendices and References: Includes supplementary material, diagrams, and reference documents used in the project.

2. SYSTEM OVERVIEW

2.1 PRODUCT PERSPECTIVE

Dixen is a standalone web application built with Angular (frontend) and ASP.NET Core Web API (backend), using a relational database (MSSQL). The system follows modular architecture and communicates via secure RESTful APIs.

2.2 KEY FEATURES

- Secure user registration with email confirmation: Users register by providing an email and password. A confirmation email is sent to validate the user's email address.
- Two-factor authentication (2FA): Users can enable 2FA during login. After entering their credentials, users are required to input a 2FA code sent to their email. This adds an extra layer of security to the login process.
- Event proposal and approval workflows: Users can propose events, and organizers can approve or reject them through the platform.
- Categorization of events (e.g., music, tech, art): Events are categorized based on themes such as music, technology, and art.
- Role-based access for users and organizers: Users can register, submit proposals, and share events, while admin have full control over event approvals and lifecycle management.
- Social sharing integration: Events can be shared on social media platforms like Facebook and Twitter.
- Performer and venue management: Host can manage performer lists and venue details.
- Analytics dashboard with visual charts: Interactive dashboards display event, registration, and performance statistics using graphical charts.
- Multi-language support: Application support for two languages, allowing users to switch languages dynamically.

2.3 PROGRAM FUNCTION

Dixen allows users to authenticate securely, browse and propose events, and manage participation. Organizers manage event details and submissions, while administrators approve events and manage system data. The platform supports analytics, role-based access, and secure communication.

2.4 OPERATING ENVIRONMENT

Client: Modern web browsers (Chrome).

Devices: Desktop, Tablet, Mobile

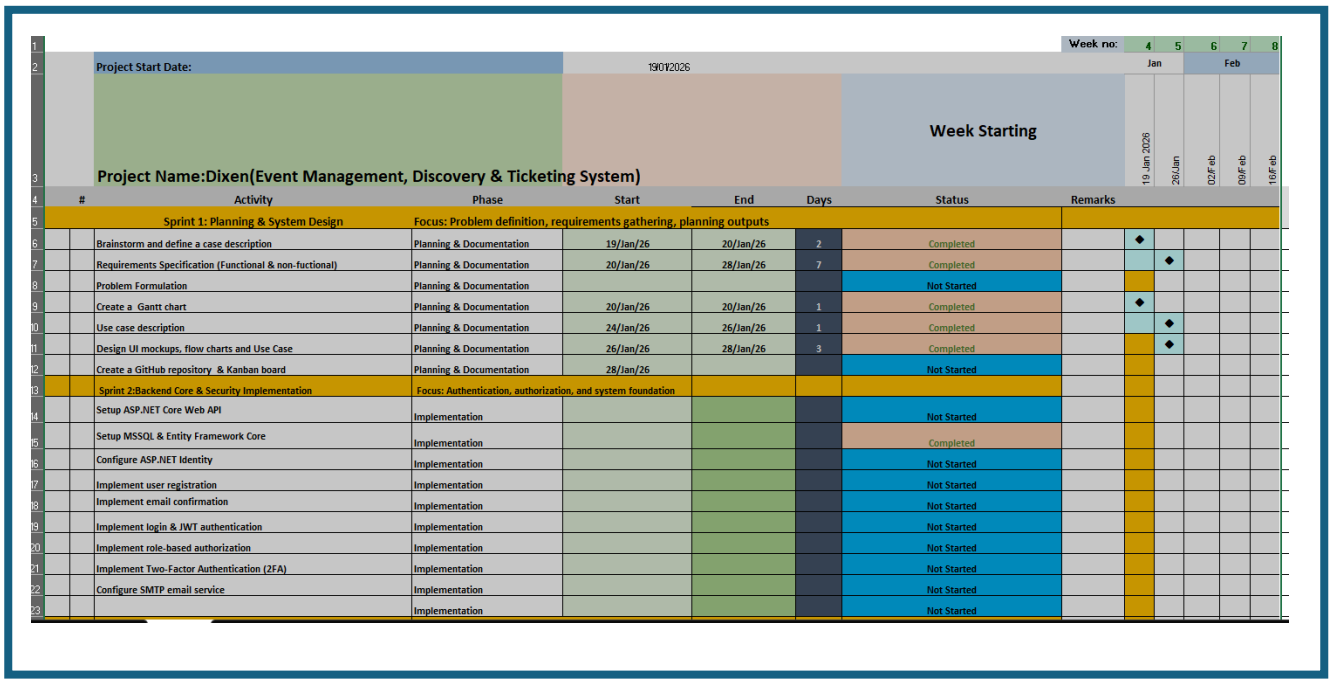
Server: ASP.NET Core Web API

Database: MSSQL database.

Authentication: JWT-based authentication with email verification and 2FA.

2.5. DEVELOPMENT METHODOLOGY

The project follows Agile methodology, with iterative weekly sprints focused on delivering incremental functionality and incorporating feedback.



		Activity	Phase	Start	End	Days	Status	Remarks
Project Start Date:			19012026					
Project Name:Dixen(Event Management, Discovery & Ticketing System)							Week Starting	
		Sprint 1: Planning & System Design	Focus: Problem definition, requirements gathering, planning outputs					
		Brainstorm and define a case description	Planning & Documentation	19/Jan/26	20/Jan/26	2	Completed	
		Requirements Specification (Functional & non-fuctional)	Planning & Documentation	20/Jan/26	28/Jan/26	7	Completed	
		Problem Formulation	Planning & Documentation				Not Started	
		Create a Gantt chart	Planning & Documentation	20/Jan/26	20/Jan/26	1	Completed	
		Use case description	Planning & Documentation	24/Jan/26	26/Jan/26	1	Completed	
		Design UI mockups, flow charts and Use Case	Planning & Documentation	26/Jan/26	28/Jan/26	3	Completed	
		Create a GitHub repository & Kanban board	Planning & Documentation	28/Jan/26			Not Started	
		Sprint 2:Backend Core & Security Implementation	Focus: Authentication, authorization, and system foundation					
		Setup ASP.NET Core Web API	Implementation				Not Started	
		Setup MSSQL & Entity Framework Core	Implementation				Completed	
		Configure ASP.NET Identity	Implementation				Not Started	
		Implement user registration	Implementation				Not Started	
		Implement email confirmation	Implementation				Not Started	
		Implement login & JWT authentication	Implementation				Not Started	
		Implement role-based authorization	Implementation				Not Started	
		Implement Two-Factor Authentication (2FA)	Implementation				Not Started	
		Configure SMTP email service	Implementation				Not Started	
			Implementation				Not Started	

3. USER REQUIREMENTS

Dixen is designed to meet the needs of individuals, organizations (hosts), or small businesses who want a simple and modern way to manage events online.

3.1 USER PROFILE

Each registered user has a profile containing personal details and activity data.

Profile Fields include:

- Full Name
- Email Address (verified)
- Role (User / Host / Admin)
- Event History (attended, hosted, proposed)
- 2FA Enabled (Yes/No)
- Preferences (categories, notifications)

The profile allows users to:

- View/edit their data
- Manage password and 2FA
- View past and upcoming events

3.2 USERS EXPECTATIONS

Customers want a platform that allows them to easily create and share events without needing technical expertise. They want to be able to set up an event by simply filling in the essential details such as title, description, time, location, and number of participants.

In addition, users expect to register and manage their attendance with minimal effort. They want to receive clear confirmations, reminders before events, and be able to access an overview of all their upcoming or past events.

Organizers also want a simple dashboard that gives them a full overview of the events they've created, how many participants are registered, and whether any updates are needed. Security and data protection are important — especially for handling participant information.

To be accessible and competitive in today's digital environment, Dixen must be fully responsive and function well on all modern web browsers. Users expect to access the platform

just as easily from a desktop computer as from a smartphone or tablet, without needing to install any app.

4. SECURITY

4.1 AUTHENTICATION

- Users register using email/password.
- Email confirmation is mandatory before logging in.
- JWT tokens are issued upon successful login.

4.2 TWO-FACTOR AUTHENTICATION (2FA)

- Optional 2FA during login.
- 6-digit code sent to user's verified email.
- Enhances security by requiring access to both password and email.

4.3 ROLE-BASED ACCESS CONTROL (RBAC)

Dixen implements a robust Role-Based Access Control system to ensure that users can only access features and data appropriate to their responsibilities. The system defines the following roles, each with distinct permission:

Role	Description	Permission
User	General platform user who can browse and attend events.	➢ Register and manage profile ➢ Browse and search events ➢ Register for events ➢ Propose events ➢ Share events on social media ➢ Manage 2FA and security settings
Host	Users responsible for creating and managing events.	➢ All User permissions ➢ Assign performers and venues ➢ View analytics dashboard ➢ Submit event proposals for admin approval
Admin	Platform administrators with full system control.	➢ All Organizer permissions ➢ Approve/reject event proposals ➢ Manage user roles and permissions ➢ Access and manage all events, users, and system settings ➢ Monitor security and logs

Access Control Rules:

- Users can only view, register for events and buy tickets.
- Hosts manage their events but require admin approval for publishing.
- Admins have unrestricted access.

Enforcement:

- Server-side role checks for API and UI access.
- Role changes are audited and restricted to Admins.

4.4 PASSWORD & SESSION MANAGEMENT

- Passwords are hashed using ASP.NET Identity with salted hashing.
- JWT tokens are used for stateless session management.

4.5 DATA PROTECTION

- All communication secured via HTTPS.
- Sensitive data (password, token) is handled securely using ASP.NET identity and JWT authentication.
- Two factor authentication (2FA) is implemented to add an extra layer of account security.
- Email for confirmation and 2FA codes are sent securely via SMPT.

5. FUNCTIONAL REQUIREMENTS

5.1 PRIORITY DEFINATION

In this project, all requirements are classified into three priority levels:

High Priority: Critical requirements that must be implemented for the system to function correctly. These are highlighted in red throughout the document.

Medium Priority: Important requirements that should be implemented but are not critical for the initial system release.

Low Priority: Optional or nice-to-have requirements that can be implemented later if time permits.

The priority level of each requirement is indicated using color highlighting in the Functional Requirements tables and Use Case descriptions.

LIST OF REQUIREMENTS SPECIFICATION

USER REGISTRATION AND LOGIN

The system shall provide secure user registration and login functionalities using ASP.NET Identity. Upon successful login, a JWT token will be issued for authenticated API access.

TWO-FACTOR AUTHENTICATION (2FA)

The system shall implement an additional security layer through two-factor authentication. A 6-digit verification code shall be sent to the user's registered email for login validation.

EMAIL COMMUNICATION

The system shall send confirmation emails, notifications, and 2FA codes using SMTP protocol. Email addresses must be validated during registration.

API DATA EXCHANGE

The backend shall expose RESTful APIs to handle business logic, user operations, and database interaction securely via JWT-based authentication.

EVENT MANAGEMENT

- Users can propose events.
- Admins approve/reject event proposals.
- Admins manage all events (create, update, categorize, and delete events).

- Events can belong to one or more categories (e.g., music, tech, art).

ROLE-BASED ACCESS CONTROL

- The system differentiates access levels for users, host & admins.

PERFORMER AND VENUE MANAGEMENT

- Organizers (host) assign performers and venues to events.

SOCIAL SHARING

- Events shall be shareable on platforms like Facebook.

ANALYTICS DASHBOARD

- Host view statistics on event attendance and registrations.

6 USE CASE DESCRIPTIONS

6.1 USER REGISTRATION WITH EMAIL CONFIRMATIOSN

Actor: User	**Id:** UC- 1
Description: The user registers for the application, and the system sends an email confirmation link.	
Precondition: The actor has a valid email address.	
Input: Full Name, Email, Password, Age and Gender	
Output: ➢ User account created in the database. ➢ Confirmation email sent to the provided email address. ➢ Users receive a message to check their email for confirmation.	
Expected Result: The user account is created in the system in an unconfirmed state, and a conformation email with a link is sent to the user's email address.	
Main Flow: ➢ The actor enters the required registration information (e.g., name, email, password). ➢ The system validates the information and creates the user's account in the database in an unconfirmed state. ➢ The system sends an email with a confirmation link to the user's email address. ➢ The system informs the user to check their email for the confirmation link.	
Alternative Flows: ➢ The user enters invalid registration information (System shows validation errors). ➢ The confirmation email fails to send (system informs the user to try again).	

6.2 CONFIRM EMAIL ADDRESS

Actor: User	**Id:** UC- 2
Description: The user confirms their email address by clicking the confirmation link received during registration. The system activates the account.	
Precondition: User has received a valid confirmation email from registration and Confirmation link is valid and not expired.	
Input: Conformation link (containing user identifier and token)	
Output: ➢ User's email address is confirmed. ➢ User's account is activated for login.	
Expected Result: The user's email is confirmed; account becomes active for login.	
Main Flow: ➢ The user clicks the confirmation link in their email ➢ The system validates confirmation requests. ➢ The system confirms the email address and activates the user's account. ➢ The System notifies the user of successful confirmation.	
Alternative Flows: ➢ The Confirmation link is invalid or expired → System shows error message. ➢ User not found → System shows error message.	

6.3 LOGIN WITH 2FA (QR CODE AND GOOGLE AUTHENTICATION)

Actor: User	**ID:** UC-3
Description: The user login to the application using email and password, followed by two-factor authentication (2FA) when required.	
Precondition: ➢ User has a confirmed email address and active account. ➢ User has 2FA setup completed (if required).	
Input: Email, Password and 2FA code.	
Output: ➢ Users are authenticated and logged in. ➢ JWT token provided for protected areas. ➢ User is authenticated and logged in and redirected to the dashboard.	
Expected Result: The user is successfully logged in with valid credentials and 2FA (if required), receiving a JWT token for session access.	
Main Flow: ➢ The user enters their registered email address and password. ➢ The system validates the credentials. ➢ If 2FA is required, the system prompts for 2FA code. ➢ The user enters the 2FA code from their authenticator app. ➢ The system verifies the code and generates JWT token. ➢ The user is logged in, and the JWT token is returned.	
Alternative Flows: ➢ Invalid email or password. The system prompts the actor to try again. ➢ Email not confirmed → System shows "Please confirm your email first" message. ➢ Invalid 2FA code → System prompts user to try again. ➢ User has no 2FA setup → System skips 2FA step and logs in directly.	

6.4 EVENT CATEGORIZATION (EVENTS CAN BELONG TO MULTIPLE CATEGORIES)

Actor: Admin	**ID**: UC-4
Description: The admin can assign an event to multiple categories.	
Precondition: The admin is logged into the application and is editing or creating an event.	
Input: ➢ Event details (e.g., title, description) ➢ Selected categories	
Output: ➢ The event is associated with the selected categories in the database.	
Expected Result: The admin can assign multiple categories to the event.	
Main Flow: ➢ The admin creates or edits an event and selects the Categories option. ➢ The system displays available categories (e.g., Technology, Art, Science). ➢ The admin selects one or more categories to assign to the event. ➢ The system saves the event's category selections and associates them with the event.	
Alternative Flows: ➢ If no categories are selected, the system prompts the organizer to select at least one category. ➢ If an invalid category is selected, the system shows an error and prompts the organizer to select a valid category.	

6.5 EVENT BOOKING

Actor: User	**ID**: UC-5
Description: The registered user books a ticket for a selected event. The system assigns the user to a hall with available capacity and sends a booking confirmation email.	
Precondition: ➢ The user is logged in to the application. ➢ Event must exist and have at least one hall with available capacity.	
Input: EventId	
Output: ➢ Booking records are created in the database. ➢ Hall Assigned ➢ Confirmation email sent to the user.	
Expected Result: The user successfully books a ticket for the selected event and receives a confirmation email with booking details.	
Main Flow: ➢ The user selects an event to book. ➢ The system verifies the user's identity from the authentication token. ➢ The system retrieves event details and hall information. ➢ The system checks hall capacities. ➢ The system assigns the user to the first available hall. ➢ The system creates a booking record. ➢ The system sends a confirmation email to the user. ➢ The user sees a booking confirmation message.	
Alternative Flows: ➢ No Available Seats: The system informs the user that the event is fully booked. ➢ User Not Authenticated: The system returns an unauthorized error. ➢ Event Not Found: The system returns an error indicating the event does not exist.	

6.6 SUBMIT EVENT PROPOSAL (EVENTSUBMISSION)

Actor: User	**ID**: UC-6
Description: The user submits a proposal to present or participate in an event.	
Precondition: The user is logged in.	
Expected Result: An EventSubmission record is saved for review.	
Input: Proposal details (e.g., EvantId, description)	
Output: ➢ EventSubmission record saved in the database. ➢ Admins notified or can view new proposals. ➢ Confirmation message displayed to the user.	
Main Flow: ➢ The user selects a ”Submit Proposal". ➢ Fill out the form with details (Id, description, etc.). ➢ The submission is saved in EventSubmissions. ➢ Admins are notified or can view new proposals.	
Alternative Flows: ➢ Incomplete or invalid information is entered. ➢ Duplicate proposal submitted.	

6.7 SOCIAL SHARING (SHARE EVENT ON PLATFORMS LIKE TWITTER, FACEBOOK)

Actor: User	**ID:** UC-7
Description: The user can share an event on social media platforms like Twitter and Facebook.	
Precondition: The actor is logged in to the application.	
Input: Select event, Chosen social media platform	
Output: ➢ Event details are shared on the selected platform. ➢ SocialShares record created in the database. ➢ A confirmation message is displayed to the user.	
Expected Result: The actor can share the event on the selected social media platform.	
Main Flow: ➢ The user clicks "Share Event". ➢ The system displays options to share the event on social platforms such as Twitter, Facebook, or others. ➢ Select a platform (e.g., Twitter). ➢ If the user is not logged in to the selected social media platform, the system prompts the user to log in before proceeding. ➢ The user logs in to the selected social media platform (if necessary). ➢ The system shares the event details on the chosen platform, including the event title, description, and a link. ➢ The system logs the share in SocialShares.	
Alternative Flows: ➢ Sharing fails due to a network error. ➢ Unsupported platform is selected.	

6.8 EVENT MANAGEMENT (CREATE, EDIT, AND DELETE EVENTS)

Actor: Admin	**ID:** UC-8
Description: The admin can create, edit, and delete events.	
Precondition: The admin must be logged into the application.	
Inputs: ➢ Event Details (e.g., Title, Description, Date/Time) ➢ Selected Categories ➢ Assigned Halls and Performers	
Output: ➢ Event Record Created/Updated/Deleted ➢ Confirmation Message Displayed ➢ Error Message if Constraints are Violated (e.g., Date Conflict, Missing Fields)	
Expected Result: The organizer can successfully create, edit, or delete an event.	
Main Flow: ➢ The admin selects the Create Event option and fills in the necessary event details (e.g., event name, date, venue). ➢ The system validates the input and creates a new event in the database. ➢ The system associates categories and performers to the event as required. ➢ The admin can choose to edit an existing event by selecting the Edit Event option. ➢ The admin updates the event information (e.g., change of venue, date, performers). ➢ The organizer can delete the event by selecting the Delete Event option. The system confirms the deletion request and removes the event from the database.	
Alternative Flows: ➢ If the admin enters invalid information (e.g., date conflict, missing required fields), the system displays an error and asks for corrections. ➢ The event cannot be deleted if there are confirmed attendees or important associated data, and the system provides a warning message.	

7. NON-FUNCTIONAL REQUIREMENTS:

These requirements describe the intended operational qualities of Dixen, focusing on security, usability, maintainability, and scalability. As this project is for demonstration purposes, actual performance metrics are not measured.

7.1 PERFORMANCE REQUIREMENTS

- The system is designed to handle multiple users concurrently, with modular APIs for efficient data access.
- API endpoints are structured to respond quickly under normal usage conditions.

7.2 SECURITY REQUIREMENTS

- Passwords are securely hashed using ASP.NET Identity with salted hashing.
- JWT tokens are used for session management and authentication.
- Two-Factor Authentication (2FA) is available and configurable per user.
- Access to sensitive endpoints is restricted by role-based authorization.
- All communication is secured via HTTPS.

7.3 USABILITY REQUIREMENTS

- The user interface is mobile-responsive and adapts to desktop, tablet, and smartphone screens.
- Accessibility considerations follow best practices to meet standard guidelines (WCAG 2.1).

7.4 RELIABILITY AND AVAILABILITY

- The system works correctly during testing; production-level uptime or backup mechanisms are not applicable since the project is not deployed.

7.5 MAINTAINABILITY AND SCALABILITY

- The application is modular, follows clean architecture principles, and separates frontend (Angular) and backend (ASP.NET Core) concerns for ease of maintenance.
- The system design supports future feature extensions and scaling if deployed in production.

8. SYSTEM DESIGN

8.1 FRONTEND STRUCTURE (ANGULAR)

Components: Login, Register, Event-List, Event-Details, Admin-dashboard.

Generic Services: A single reusable Angular service is implemented to handle all backend API communication across components. This generic service abstracts HTTP operations (GET, POST, PUT, DELETE) and allows components to interact with different REST endpoints by specifying the endpoint URL and data payload dynamically.

8.2 BACKEND STRUCTURE (ASP.NET WEB APPLICATION)

Controllers: AuthController, EventController, EventController,ProposalController.

Services: JWTService, EmailService, 2FAService.

dentity: ASP.NET Core Identity for user management.

Data Access: Generic and non-Generic Repository, ApplicationDbContext.

Data Transfer Objects or Mappers, for clean architecture.

9. TECHNICAL PRODUCT DOCUMENTATION

9.1 DATABASE DESIGN

Database: Microsoft SQL Server.

Key entities: Users, Events, Categories, EventReview, EventSubmission, organizer(host), Tickets, Booking, Venue, Halls, Performers, Social Shares.

Design Principles: Normalized tables, indexed on key search fields, secure password storage.

9.2 EXTERNAL INTERFACES

Frontend: Angular communicates with backend via REST APIs.

Backend: It interacts with MSSQL via Entity Framework.

Email: It is sent via SMTP server.

9.3 TECHNOLOGIES USED

Frontend: Angular **20.3.15** with TypeScript **5.8.3**, HTML5, Tailwind CSS **4.1.18**

Backend: ASP.NET Core Web API **(.NET 8),** Entity Framework Core (EF Core), MSSQL.

Authentication: ASP.NET Identity with JWT-based session management.

Email Integration: SMTP for sending confirmation emails and 2FA codes.

Data visualization: AG-Charts **13.0.1** (Angular)

Internationalization: Angular i18n/ ngx-translation **17.0.0** (multi language support)

Version Control: GitHub.

Communication & Notification: Email (for sending project-related updates, reports, etc.)

10. AUTHENTICATION & AUTHORIZATION FLOW

10.1 USER REGISTRATION (VIA /API/AUTHENTICATION/REGISTER):

- The user provides their email, password, and full name.
- The system validates the email format and checks for duplicates.
- Upon successful registration, an email confirmation token is generated and sent to the user's email address.
- The user must confirm their email to activate the account.

10.2 EMAIL CONFIRMATION (VIA /API/AUTHENTICATION/CONFIRM-EMAIL):

- The user clicks the confirmation link in the email.
- The system decodes and validates the confirmation token.
- Once validated, the user's email status is marked as confirmed, and the account becomes active.

10.3 USER LOGIN (VIA /API/AUTHENTICATION/LOGIN):

- The user provides their email and password to log in.
- The system verifies the credentials and checks if the email is confirmed.
- If Two-Factor Authentication (2FA) is enabled, a one-time code is sent to the user's email.
- The user must enter the 2FA code to complete the login process.
- Upon successful login, a JWT token is issued for secure session management.

10.4 TWO-FACTOR AUTHENTICATION (2FA)

- Dixen implements email-based 2FA to enhance account security.
- After successful credential verification, a 6-digit code is sent to the user's email.
- This second layer of verification ensures that only the email owner can access the account, even if login credentials are compromised.

11. FUTURE DEVELOPMENT & INTEGRATIONS

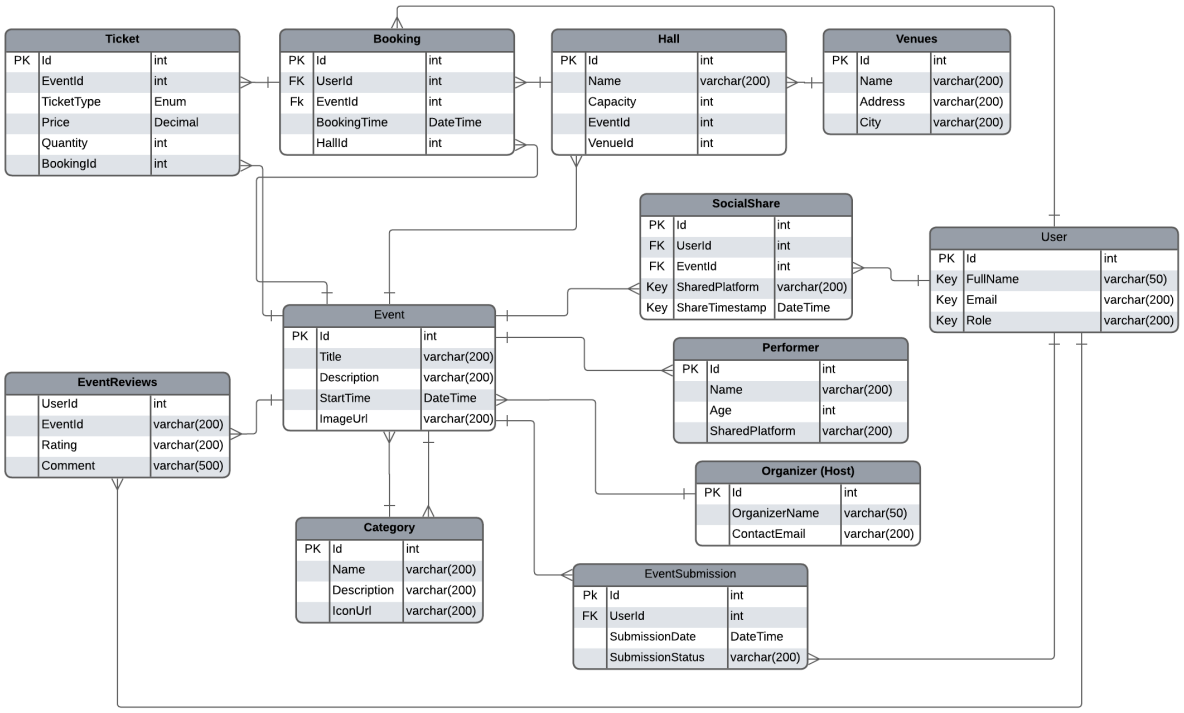
Planned enhancements include:

- Payment Gateway Integration (MobilePay)
- Push Notifications
- Google Calendar
- Business Analytics Dashboard

12. VISUAL DISGRAMS

12.1. ER-DIAGRAM

The ER diagram illustrates the main entities and relationships of the Dixen Event Management System. Users can register, book tickets, review events, and share events on social media. Events are associated with Categories, Halls, Venues, and Performers. Organizers submit events via Event Submissions, which are tracked for approval. Bookings link Users to Tickets for specific Events held in Halls. The diagram captures these interactions, showing how different entities relate to one another to support event discovery, management, and participation.



https://lucid.app/lucidchart/f7d9d136-137d-4366-9f93-85ddccdea660/edit?viewport_loc=-535%2C-303%2C2828%2C1339%2C0_0&invitationId=inv_395aea83-71a9-4996-911f-5be778701006

12.2 USE CASE DIAGRAM

This use case diagram illustrates the main interactions between guests, registered users, and administrators within the event management system. It defines core functionalities such as registration, authentication, event browsing, event proposal, and administrative approval. Include and extend relationships are used to model mandatory and optional system behaviors.



https://lucid.app/lucidchart/f7d9d136-137d-4366-9f93-85ddccdea660/edit?viewport_loc=-540%2C13%2C1865%2C1049%2CvrX5dHIHi3wb&invitationId=inv_395aea83-71a9-4996-911f-5be778701006

12.3 FLOW CHART

12.3.1. USER REGISTRATION

This diagram shows how a user signs up and verifies their account:

I. Start

- The flow begins when a user initiates the registration process.

II. Enters Registration Info

The user fills out a form with personal details like:

- Name
- Email address
- Password

III. Is it valid? (Validation Check)

The system checks whether the input is valid:

- Are all required fields filled?
- Is the email format correct?
- Are passwords matching?

If not valid, the process ends (or loops back to re-enter data).

IV. Create in Database

- If input is valid, the user's details are saved into the system's user database as an "inactive" or "unverified" account.

V. Generate Email Confirmation Token

- The system generates a unique token (a long random string) to verify the user's identity.
- This token will be embedded in a link.

VI. Send Confirmation Email with Link

- An email is sent to the user that contains the confirmation link like:
- https://example.com/verify?token=XYZ123...

VII. User Clicks Link

- The user opens the email and clicks the verification link.

VIII. Is it a valid token?

- The system checks if the token is valid:
- Is it unexpired?

- Does it match what was issued to that email?

If not valid, registration fails.

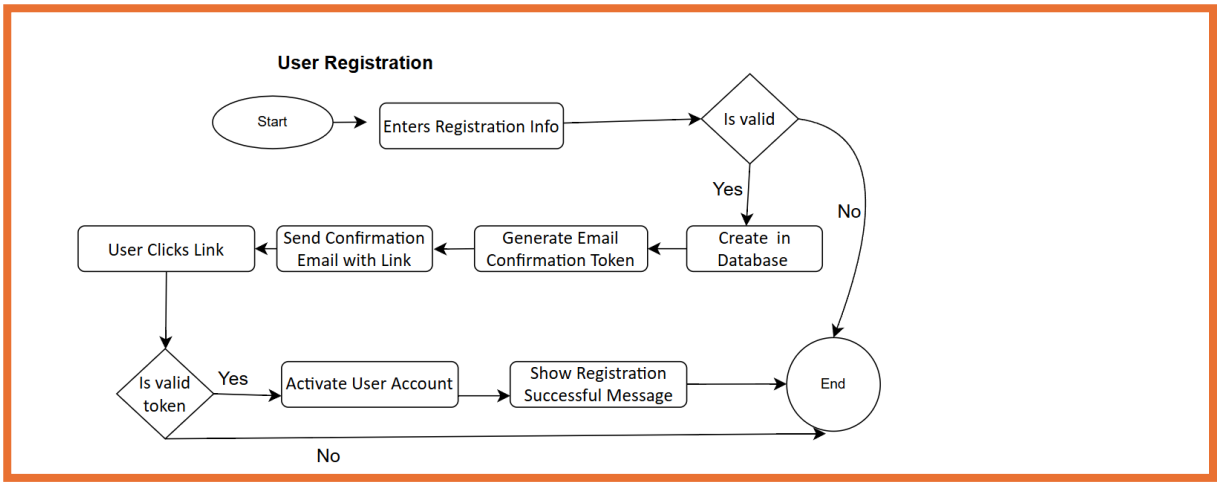
IX. Activate User Account

- If the token is valid, the system marks the user as "verified" or "active" in the database.

X. 10. Show Registration Successful Message

- A confirmation message is displayed, such as:

"Your account has been successfully registered and verified!"



12.3.2. EMAIL VERIFICATION

This diagram provides a simplified view focusing on the email verification process during user registration.

I. Start: The process begins when a user submits the registration form.

II. Registration Form Validation

The system validates the submitted form data.

- If the input is invalid, the user is prompted to correct the form and resubmit.
- If the input is valid, the process continues.

III. Send Email Confirmation Link

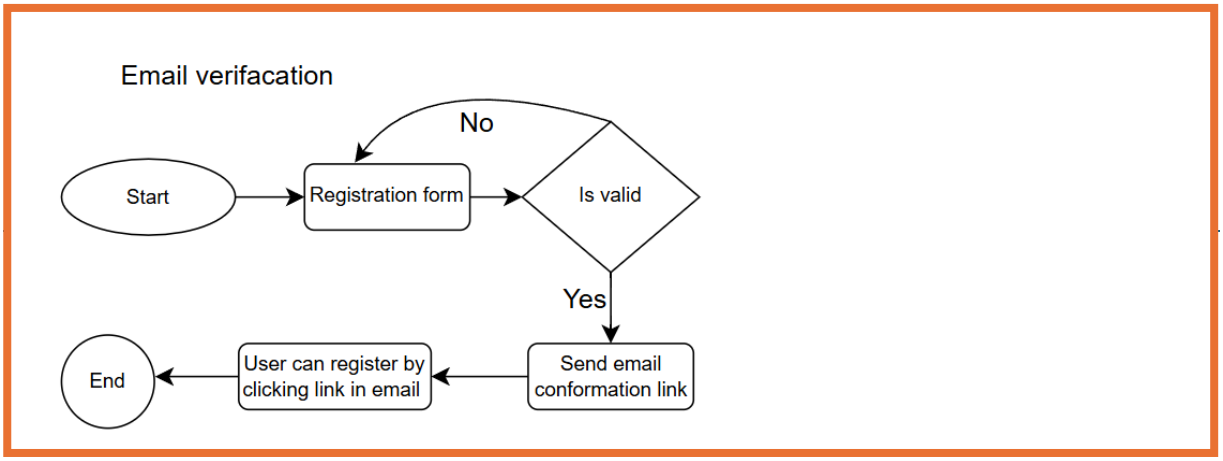
The system creates a verification token and sends a confirmation email containing a unique activation link to the user's email address.

IV. Email Confirmation

The user clicks the confirmation link in the email. The system verifies the token and confirms the user's email address, activating the account.

V. End

The flow ends after the user is successfully verified and registered.



12.3.3. LOGIN PROCESS

Once verified, the user can log in using their credentials (typically email and password). The login process use two-factor authentication to enhance security.

I. Start

The process begins when a user attempts to log in to the system.

II. Enter Username and Password

The user provides their:

- Username (or email)
- Password

This is the first layer of authentication (something the user knows).

III. Login Verification

The system checks if the provided username and password match the records in the database:

- If credentials are invalid, the login is denied and the process ends (or loops back to retry).
- If credentials are valid, the system moves to the second factor: verification code generation.

IV. Generate Verification Code (TOTP)

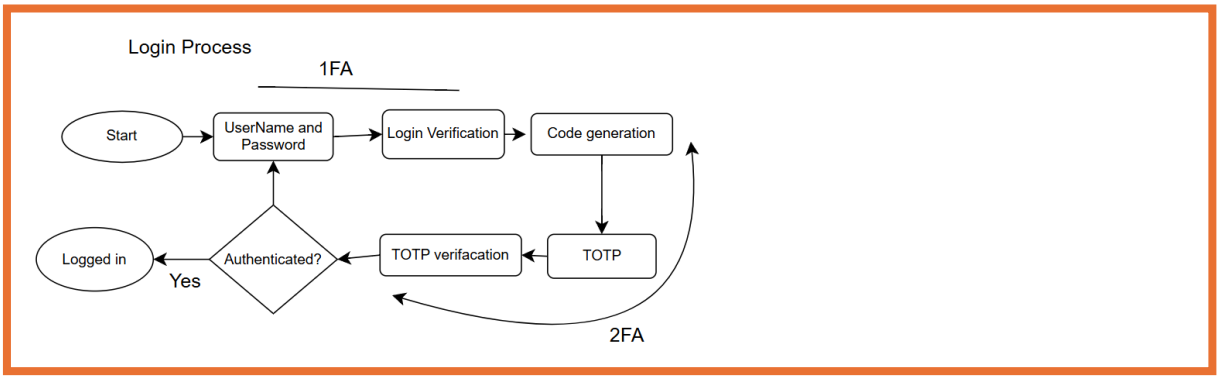
The user opens a mobile authenticator application such as Google Authenticator. The app generates a time-based one-time password (TOTP) using a shared secret key and the current time in 30-second intervals. A unique 6-digit code is generated for each time window

V. TOTP Input and Verification

- The user enters the generated TOTP code into the system.
- The backend verifies the code using the same shared secret and time window.
- If the code is valid, authentication succeeds.
- If the code is invalid or expired, an error is returned and the user is prompted to retry

VI. Authenticated Login

Once both factors are verified (password and TOTP), the system grants full access and issues a JWT token. The user is now logged in and redirected to the dashboard or home page.



12.3.4. CODE GENERATION

After logging in, the system may generate a unique code.

I. Start

The process begins when the user opens the Google Authenticator app (or similar TOTP app).

II. Open Google Authenticator

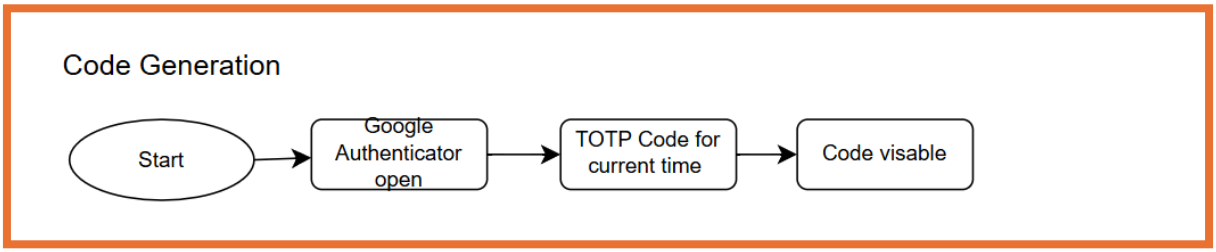
During the 2FA setup, a shared secret key is generated by the system and securely stored on both the server and the user's authenticator application. This shared key is used to generate and validate one-time codes.

III. Generate TOTP Code

The authenticator application generates a one-time password by combining the shared secret key with the current system time, divided into 30-second intervals. The application applies the HMAC-SHA1 hashing algorithm to produce a unique 6-digit TOTP code. A new code is generated every 30 seconds.

IV. Code Becomes Visible

The generated TOTP code is displayed to the user along with a timer indicating its validity period. The user enters this code into the system, where it is verified by the backend to complete the authentication process.



12. REFERENCES

ASP.NET Core 8 Documentation**:** https://learn.microsoft.com/en-us/aspnet/core/

Angular Documentation: https://angular.io/docs

SMTP: Simple Mail Transfer Protocol:

https://www.cloudflare.com/learning/email-security/what-is-smtp/

OWASP Password Storage Cheat Sheet:

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Tailwind css: https://tailwindcss.com/docs/installation/tailwind-cli

Gantt: https://www.youtube.com/watch?v=tuQpGzL_14g

AG Chart: https://www.ag-grid.com/charts/angular/quick-start/